R Practice Project 1

Joseph Tarlin

College of Professional Studies, Northeastern University

Aly 6000 Introduction to Analytics

Professor Steward Huang

April 15, 2026

The purpose of Aly6000 Module 1 Project was to become familiar working with the different functions in the R Studio platform. The practice problems created different vectors which were then assigned to variables to store, remove data points from, or add data into other previously created variables using the different commands in R Studio. For example, the basic operators began with writing code to computer 123 * 453 = 55,719 in R, which is just basic multiplication. 5 ^ 2 * 40 refers to five squared multiplied by forty which gives us 1,000. The code TRUE & FALSE yields a FALSE result because both sides are not true, the & symbol means that it is logical only if both sides are true. On the other hand, TRUE | FALSE yields a TRUE result because the "|" symbol shows true if either side is true. The function 75 %% 10 is essentially asking what is left over when 75 is divided by 10, which would be 7 with a remainder of 5. 75 / 10 in R Studio is basic division to yield a result of 7.5.

After the basic R Studio computation commands were established, the code moved into creating vectors that were being stored as variables. Using the c function, we created the function first_vector <- c(17, 12, -33, 5) which assigned these four values to this first vector variable. Moving forward, another vector called counting_by_fives <- c(5, 10, 15, 20, 25, 30, 35) was created in increments of fives as it was stored in that variable. A variable called second_vector <- 20:1 was created which contains the numbers twenty to one and is counting down from twenty. After this, counting_vector <- 5:15 was established to store the results containing the numbers from five to fifteen.

Through working with vectors, the focus of the project shifted to analyzing a dataset of grades where a variable called grades <- c(96, 100, 85, 92, 81, 72) to demonstrate a set

of grades received by a student. In the event of bonus_points_added <- c(99, 103, 88, 95, 84, 75) this variable was added to evaluate the grades if the student received three extra bonus points on each score. A vector of one_to_one_hundred <- c(1,100) was created to store the values 1-100 into this specific variable. The second_vector variable was then analyzed through different operational functions on the R Studio platform – for example, the code second_vector + 20 takes the second_vector <- 20:1 variable and adds twenty to each data point which yields the values 40, 39, 38, 37, 36, 35, 34, 33, 32, 31, 30, 29, 28, 27, 26, 25, 24, 23, 22, 21. second_vector * 20 takes that variable c<- 20:1 and multiplies every value in that by twenty to get the values 400, 380, 360, 340, 320, 300, 280, 260, 240, 220, 200, 180, 160, 140, 120, 100, 80, 60, 40, 20. The function second_vector >= 20 which produces TRUE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE. The function second_vector ! = 20 refers to which elements are not equal to the value twenty in the second_vector variable. This would yield the result of FALSE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE.

The project then shifted to the focus of analyzing the dataset in different methods of looking at data such as mean, median, minimum, and maximum. The variable total <- sum (one_to_one_hundred) was commanded to compute the sum of these values which equaled 5,050. The mean function was used to compute the average_value <- mean(one_to_one hundred) which got us the average value of 50.5. The median_value <- median (one_to_one_hundred) was also used to get the median (middle value) of this data

set involving the numbers one to one hundred, which was 50.5. The max_value <- max (one_to_one_hundred) which was 100 and the min_vaue <- min (one_to_one_hundred) which is 1 were both stored as variables using the datapoints with numbers 1-100 to talk about the minimum and maximum values of the already established one_to_one_hundred variable.

Shifting to the usage of brackets in R Studio, the function first_value <- second_vector [1] used the one in the brackets to extract the first value from this second_vector variable we already created, and it gives us 20 as a result. Using the same bracket idea, we can extract the first three values by using the variable first_three_values <- second_vector [1:3] to remove the first three values from this variable and it gives us the values 20, 19, 18. In order to extract the first, fifth, tenth, and eleventh value from the second_vector, the variable vector_from_brackets <- second_vector [c(1, 5, 10, 11)] is used to get the values 20, 16, 11, and 10 respectively. The vector_from_boolean_brackets <- first_vector [c(FALSE, TRUE, FALSE, TRUE)] takes the values from the variable first_vector and true essentially means to keep that element whereas false means to drop that element – so the values 12 and 5 are kept whereas 17 and -33 are dropped.

While examining code the problem second_vector >=10 appeared which shows us the results from that previously created second_vector that are either equal to or greater than the value 10 – the respectively values from this R code end up being TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE because the first 10 values of that second_vector are all greater than or equal to 10 (true), and the last 10 values of the second_vector variable

are not equal to 10, so therefore they are false. The code one_to_one_hundred [one_to_one_hundred >= 20] only includes the elements in that variable that are greater than or equal to 20, which would include values between 20 and 100.

In order to further analyze the dataset, we removed certain grades, starting with the lowest_grades_removed <- grades [grades >85], which only included the grades the student got above an eighty-five. This would be the grades of 96, 100, and 92. The middle grades of the dataset were then removed using the variable middle_grades_removed <- grades [-c, (3, 4)] – this essentially took out the third and fourth elements of the previously established grades vector to give us values of 96, 100, 81, and 72. Continuing with removing elements of second_vector, a variable called fifth_vector <- second_vector [-c(5, 10)] was created to include every single value besides the values 16 and 11 by removing the fifth and tenth elements of this second_vector variable.

The next problems use the newly created code “set.seed (5) (skip to next line) random_vector <- runif (n=10, min = 0, max = 1000)” which focuses on the random_vector variable as a dataset that generates 10 random numbers between the values 0 and 1000, but keeps these numbers the same for data analysis purposes. Using the same sum function as before, we use the variable sum_vector <- sum(random_vector) to compute the total of that random_vector code – so essentially the 10 randomly selected datapoints between 0 and 1000 are summed together, which gets us a result of 5295.264. The variable cumsum_vector <- cumsum(random_vector) is then used to compute the cumulative sum of the random_vector datapoints which gives us the values of 200.2145, 885.4330, 1802.3088, 2086.7083, 2191.3584, 2892.4159, 3420.3759, 4228.3111, 5184.8112, and

5295.2642. Continuing to use central tendency metrics involving the random_vector in R Studio, the variable mean_vector is commanded using the mean function with the code mean_vector <- mean(random_vector) of the data involved in that random_vector variable to get the average of those points being 529.5264. The standard deviation which is the statistic that shows the dispersion of the data compared to the average of the overall dataset is coded in R Studio using the variable sd_vector <- sd(random_vector) – this shows us the variance in data amongst the values generated from that random_vector variable and it is right around 331.3606. The function round_vector <-round(random_vector) helps to round the values in the random_vector variable so that there are not very complex decimals and we are rather dealing with integers. These numbers round up or down to 200, 685, 917, 284, 105, 701, 528, 808, 957, 110. Lastly, we can use the variable sort_vector <- sort(random_vector) which essentially arranges the values in the vector from smallest to largest – so the values of n are represented by 105, 110, 200, 284, 528, and more larger numbers, which make up that random_vector variable dataset.

The last part of this R Practice for Project 1 was to download a datafile ds_salaries.csv and which analyzes different professions. Specifically, it looks at the work year, experience level, employment type, job title, salary, type of salary currency, what the salary is worth in USD, employer residence, remote ratio, company location, and the company size. The function read.csv makes it so that we were able to take this file and bring it into R Studio as a data frame so that we can analyze and edit it. The variable first_dataframe <- read.csv ("ds_salaries.csv") was used to give us 607 observations of the 12 different variables. It provided different options for each variable such as the currency

between EUR, USD, JPY, and even more choices. The remote ratio choices varied for each individual data point between 0%, 50%, and 100% meaning fully in person, a hybrid schedule, or a fully remote schedule. The variable summary (first_dataframe) was used to produce the summary statistics based on each column of this data frame. For example, the salary summary statistics showed that the minimum was 4,000, the 1st quartile was 70,000, the median was 115,000, the mean was 324,000, the third quartile was 165,000, and the maximum was 30,400,000. However, this was just salary in terms of the currency respective to how they are getting paid. If we did it in USD, the minimum would be $2,859, 1st quartile $62,726, median $101,570, mean $112,298, 3rd quartile $150,000 and the maximum being $600,000 – these are vastly different numbers, so it really does depend on the category being analyzed.

To ultimately summarize and interpret the results this project was mainly geared to get use to the variables, vectors, and other functions/commands that will be used in the R Studio platform. We were able to see how removing or using summary statistic commands in R Studio can impact the analysis of a student's grades on their exams over the course of a class they are taking. We then analyzed a CSV file which shows us how to bring data into R Studio and look at the results – from this it was concluded that R Studio can provide summary statistics for different professions, but that the individual analyzing the statistics must be careful to consider which category they are looking at (huge difference between salary vs salary in USD) to put all of the data provided into perspective.

References

IDRE Statistical Consulting Group. (2026). *Introduction to R.* ORAC Stats UCLA.

https://stats.oarc.ucla.edu/stat/data/intro_r/intro_r_interactive_flat.html

Kabacoff, R. I (2022). *R in Action: Data analysis and graphics with R and tidyverse (3rd ed.)*.

Manning Publication. ISBN 978-1-617-29605-5